Datalog

Classification of query languages

- For the **RELATIONAL** data model

FORMAL languages

- **Relational Algebra**
- Relational Calculus
- Logic programming (**Datalog**)

"COMMERCIAL" languages

- SQL (Structured Query Language)
- QBE (Query By Example)

Some Limitations of Relational Algebra

With our (very simple) algebra:

- We can extract data items, but no new value can be derived (computed) from those in dataset
  - At tuple- or value- level: no arithmetics, no unit conversions, no recombining of any value
  - At table- or column- level: no aggregates, no count, no averge, …
- We have existential quantification and negation, but not recursion (*cannot express the transitive closure of a relation*)

→ b.t.w., SQL has all of these extensions

Transitive closure of a relation

Find all hierarchical relationships (for each employee, list all the bosses)

Direct Supervision

Boss	Employee
Falchi	Rossi
Cervi	Neri
Leoni	Falchi
Galli	Leoni



Indirect Supervision

Boss	Employee
Falchi	Rossi
Cervi	Neri
Leoni	Falchi
Galli	Leoni
Leoni	Rossi
Galli	Falchi
Galli	Rossi

Shall we try a transitive closure in RA?

For each employee, find the boss and the boss's boss

Boss and Boss's boss

To find the «boss's boss's boss»… three joins…

… a new, dedicated expression should be added for each level of depth considered in the hierarchy

A logic-based query language

- **Datalog:** a sort of "Prolog for databases" (1984)
- Prolog: A logic programming language (1970)
  based on *rules*
- Rules define new «views» over existing data
  - New knowledge is inferrred from known facts

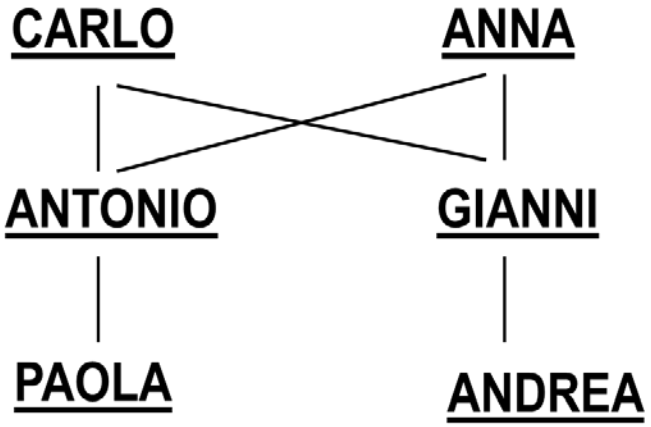
Example dataset

PARENTHOOD

Parent	Child
Carlo	Antonio
Carlo	Gianni
Anna	Antonio
Anna	Gianni
Gianni	Andrea
Antonio	Paola

PERSON

Name	Age	Sex
Carlo	65	M
Antonio	40	M
Anna	60	F
Gianni	43	M
Andrea	22	M
Paola	20	F



A Datalog program is a set of rules

A rule is made of **predicates** p_i (or **literals**)

Each rule has a **head** (or LHS) and a **body** (or RHS)

$$p_h :- p_{b1}, p_{b2}, \dots p_{bn}$$

Each ***literal*** is an *instance* of a predicate with:

- a name
- a list of arguments (within brackets), that can be:
  - **constants**
  - **variables**
  - **"*don't care*"s** (denoted as _): special variables than can only occur in the body

example of literal: Student(X , _ , "Milan", _)

Rules

A rule is interpreted as an inference:

If the body is true, then the head is also true

Example:

Father(X,Y) :- Person(X, _ , 'M'), Parenthood(X, Y)

Safety rule: all variables of the head must occur in at least one literal of the body

Rules and datasets

Each tuple in the DB represents a **basic fact** (or *ground literal*), denoted as a predicate with only constants

Examples of ground literals:

Parenthood("Carlo","Antonio")

Person("Carlo" , "65" , "M")

These «facts» are stated in the first tuples of our dataset.

A ground literal is either **true** or **false** (if a corresponding tuple **is** or **is not** in the DB)

Interpretation of Rules

In a rule:

Father(X,Y) :- Person(X, _ , 'M'), Parenthood(X, Y)

- *the LHS (head) is **true** if RHS (body) is **true***
- *the RHS is **true** if (informally) all its literals are **true***
  - More precisely, if all the variables in the body literals are ***unifiable*** (i.e., it is possible to assign them with consistent costant values taken from the facts in the DB)

Person(X, _ , 'M') has 4 possible unifications for ***X*** :

{"Carlo", "Antonio", "Gianni", "Andrea"}

Parenthood(X,Y) has 6 possible unifications for ⟨***X,Y***⟩ :

{ ("Carlo","Antonio"), ...} (the whole PARENTHOOD relation)

Interpretation of Rules

X → {“**Carlo**”, “**Antonio**”, “**Gianni**”, “**Andrea**”}

Therefore, *X* and *Y* in *Father* have the following possible unifications:

{ (“Carlo”, “Antonio”),
(“Carlo”, “Gianni”),
(“Gianni”, “Andrea”),
(“Antonio”, “Paola”) }

PARENTHOOD

Parent	Child
Carlo	**Antonio**
Carlo	**Gianni**
Anna	**Antonio**
Anna	**Gianni**
Gianni	**Andrea**
Antonio	**Paola**

Comparisons and functions

In the body of a rule we allow «**special predicates**» that are not literals, but comparison predicates

- operatori di confronto: $\neq, <, \leq, >, \geq$
- $=$ is also available, but normally expressed by unification

Example: ***Adult(N) :- Person(N, A, _), A*** $\geq$ ***18***

In the head of a rule we also allow **functions**

- Potentially any function, but we restrict to arithmetic operators : $+, -, *, \div$

Example: ***AgeNextYear(N,A+1) :- Person(N, A, _)***

Other rules on the same DB

Mother(*X*,*Y*) :- *Parenthood*(*X*,*Y*), *Person*(*X*,_,'F')

Grandparent(*X*,*Z*) :- *Parenthood*(*X*,*Y*), *Parenthood*(*Y*,*Z*)

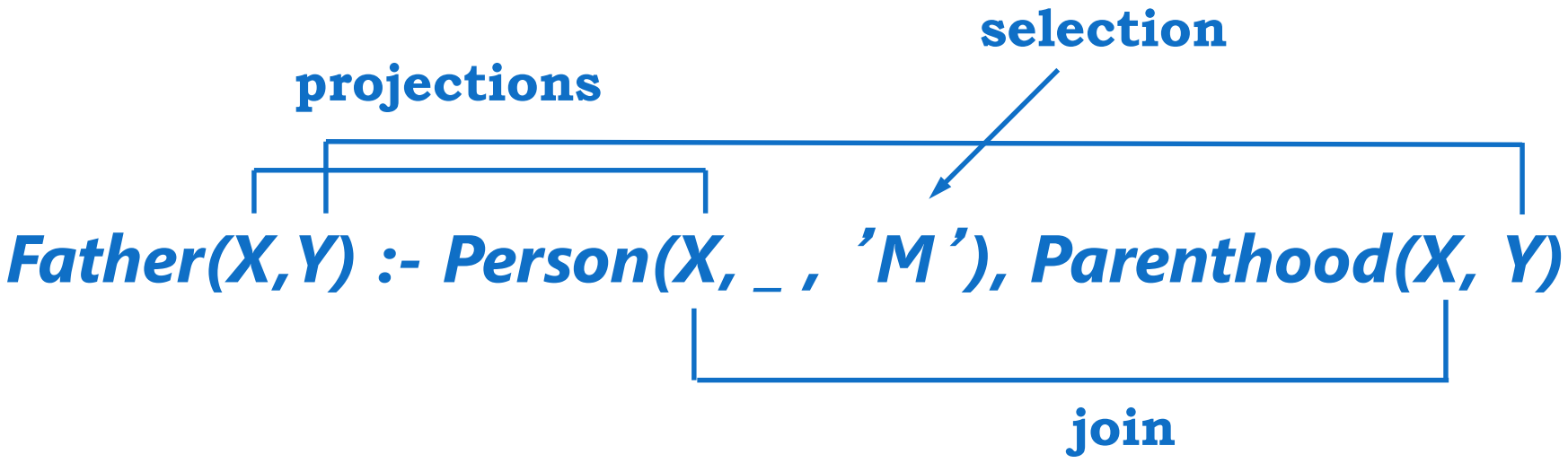
*Gram**pa**(X,Z) :- Father(X,Y), Parenthood(Y,Z)*

*Gran**ma**(X,Z) :- Mother(X,Y), Parenthood(Y,Z)*

Sibling(*X*,*Y*) :- *Parenthood*(*Z*,*X*), *Parenthood*(*Z*,*Y*), $X \neq Y$

Uncle(*X*,*Y*) :- *Person*(*X*,_,'M'), *Sibling*(*X*,*Z*), *Parenthood*(*Z*,*Y*)

How does this relate to Relational Algebra?



Corresponding expression in RA:

$$\text{Father} := \Pi_{1,5}\, \sigma_{3 = \text{"M"}} (\text{PERSON} \bowtie_{1 = 4} \text{PARENTHOOD})$$

where attribute names are replaced by the positions of the variables in the body (1 is *Name*, 3 is *Sex*, 5 is *Child*, ...)

Extensional and Intensional DBs

Extensional Database (EDB): the «basic facts» in the DB

Intensional Database (IDB): the heads of all the rules

→ *i.e.,* ***knowledge*** *that can be* ***inferred*** *from the* ***EDB***

EDB: necessarily stored somewhere

IDB: possibly generated (derived, computed, inferred)
only when necessary

Normally EDB ∩ IDB = ∅

Queries in Datalog

A query is expressed as a ***"goal"***
Who are Anna's kids?

? - Parent("Anna", X)

- All unifications for X are attempted, and the available variable bindings define the query answer
- We get *X = "Antonio"* and *X = "Gianni"*

PARENTHOOD

Parent	Child
Carlo	Antonio
Carlo	Gianni
Anna	Antonio
Anna	Gianni
Gianni	Andrea
Antonio	Paola

A goal without variables returns *True* or *False*

?- *parenthood*("Anna","Antonio") ⇒ *True*

?- *parenthood*("Anna","Andrea") ⇒ *False*

Queries in Datalog

All queries are formulated as goals
Also those that combine EDB and IDB

We can, therefore, also query the «rules», and not only the «facts»

Who's the father of Gianni? :

? - Father(X, "Gianni ")

→ X = Carlo

Queries in Datalog

Complex queries often require the definition of ad-hoc rules
We first define a dedicated intensional predicate and then a goal is expressed on that predicate.
Example: ***Find Antonio's brothers:***

→ I first define the concept of "brother"

Brother(X,Y) :- Parenthood(Z,X), Parenthood(Z,Y),
Person(Y,_,'M'), X≠Y

→ And then express the goal on the new concept

? - Brother("Antonio", Y) → Y = Gianni

Expressivity of Datalog

- Datalog (without negation) easily expresses σ, Π, $\times$, and $\cup$. The first three were already demonstrated
- **Union** is expressed by multiple definitions of the same concept (two rules with the same head)
  - $P = R \cup S$:

$$\mathbf{P(X,Y)} \text{ :- } R(X,Y).$$
$$\mathbf{P(X,Y)} \text{ :- } S(X,Y).$$

- **Difference** is expressed by negating literals
  - $P = R - S$:

$$\mathbf{P(X,Y)} \text{ :- } R(X,Y), \neg S(X,Y)$$

Negation is «dangerous» and must be disciplined

Negation in Datalog

- Literals in the body can be negated
- Negation extends the expressive power of the language
- We should be cautious in using negation
  - risk of nontermination:

$$q(X) :- \neg\, p(X) \qquad X \in \mathbb{N}$$

$$p(1)$$ *assume that p(0) is the only known fact*

… but the domain of X is infinite… almost all numbers are“non p”

? - q(X) returns an infinite result !

Usage of negation that potentially leads to nontermination is said to be **unsafe**

Safe rules (1/2)

Rules must be *well formed*:

- Negation must be *safe*:

  All variables that occur in a negated literal must also occur in a positive literal in the rule body

  S(X) :- ¬ R(X) **not** OK

  S(X) :- ¬ R(X), P(X) OK

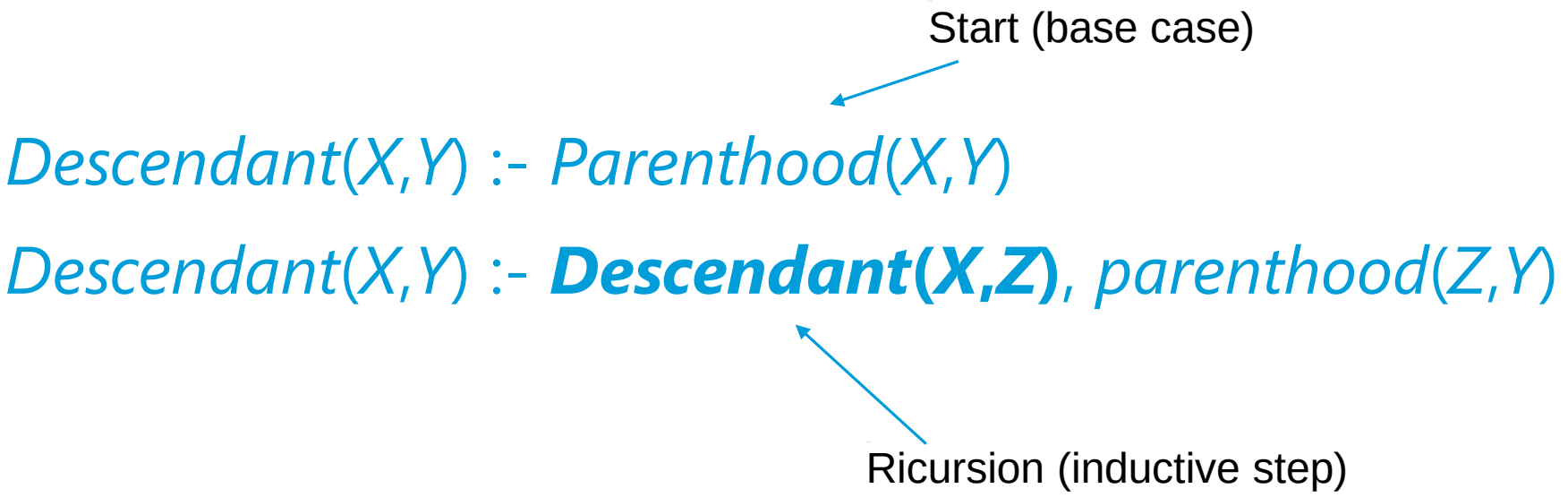
Expressivity of Datalog with negation

Datalog with negation is therefore at least as expressive as Relational Algebra
Adding negation, all basic algebraic operators are expressible in Datalog

Actually Datalog has a greater expresive power, as it also supports **recursion**

Recursion in Datalog

A recursive rule is a rule that re-uses its head as part of its own definition in the body
If it is immediately recursive, the same literal is in both the body and the head:



Evaluation mechanism

An iterative evauation procedure is executed: each iteration unifies the variables in the rules and adds (via union) the new results to the ones already discovered.

The result thus grows…. up to a certain piont.

$\mathbf{Descendant^0 \Leftarrow PARENTHOOD}$

$\mathbf{Descendant^1 \Leftarrow [(\Pi_{1,4}\ (Descendant^0 \bowtie_{2=3} PARENTHOOD\ )] \cup Descendant^0}$

$\mathbf{Descendant^2 \Leftarrow [\ \Pi_{1,4}\ (Descendant^1 \bowtie_{2=3} PARENTHOOD\ )] \cup Descendant^1}$

…

this goes on until no new knowledge can be added, and

$\mathbf{Descendant^n}$ is equal to $\mathbf{Descendant^{n-1}}$ (a **fixed point** is reached)

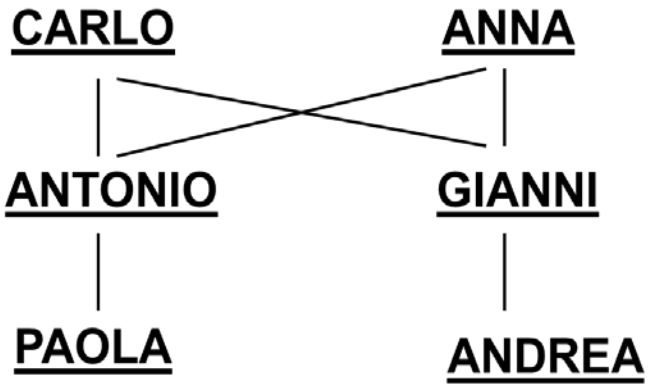
Example dataset

PARENTHOOD

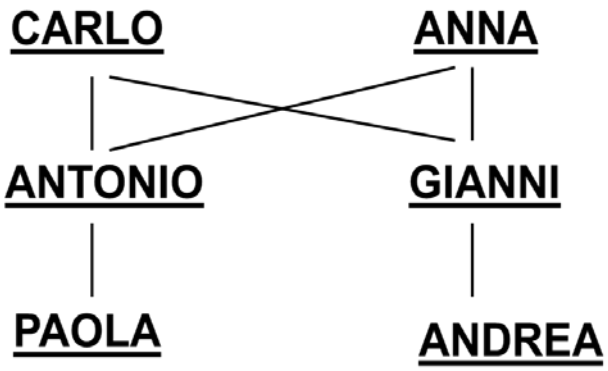
Parent	Child
Carlo	Antonio
Carlo	Gianni
Anna	Antonio
Anna	Gianni
Gianni	Andrea
Antonio	Paola

PERSON

Name	Age	Sex
Carlo	65	M
Antonio	40	M
Anna	60	F
Gianni	43	M
Andrea	22	M
Paola	20	F



Recursive evaluation



PARENTHOOD

Descendant

Carlo	Antonio	a
Carlo	Gianni	b
Anna	Antonio	c
Anna	Gianni	d
Gianni	Andrea	e
Antonio	Paola	f
Carlo	**Andrea**	**g** :- b, e
Carlo	**Paola**	**h** :- a, f
Anna	**Andrea**	**i** :- d, e
Anna	**Paola**	**j** :- c, f

New knowledge

Typical uses of Datalog: Views and Constraints

It is often convenient to define **constraints** in the form of rules that extract examples of incorrect instances

Examples:

incorrectdb1(X,Y) :- Mother(X,Y), Mother(Y,X).

incorrectdb2(X) :- Descendent(X,X).

GoT(X,Y, C) :- Siblings(X,Y),Parenthood(X,C),
Parenthood(Y,C)

If the evaluation of these rules doesn't return empty results, the incorrect data is readily available

Other recursive definitions

Supervisors and subordinates

supervisor(X,Y) :- reports_to(X,Y)

supervisor(X,Y) :- reports_to(X,Z), supervisor(Z,Y)

subordinate(X,Y) :- supervisor(Y,X)

incorrectdb1(X) :- supervisor(X,X)

incorrectdb2(X,Y,Z) :- reports_to(X,Y), reports_to(X,Z), Y ≠ Z

Other recursive definitions

- **Cousins of the same generation (not removed)**

  sgc(X,X) :- Person(X, _, _)

  sgc(X,Y) :- sgc(Z,W), Parenthood(Z,X), Parenthood(W,Y)

- **Parts of complex aggregates**

  component(X,Y):- part_of(X,Y)

  component(X,Y):- part_of(X,Z), component(Z,Y)

Problems with negation and recursion

A naive application of the evaulation procedure for recursive rules may lead to problems

Example:

Descendant(X,Y) :- Parenthood(X,Y)

Descendant(X,Y) :- Descendant(X,Z), Parenthood(Z,Y)

NonDescendant(X,Y) :- Person(X,_,_), Person (Y,_,_), ¬ Descendant(X,Y)

- At the first iteration, NonDescendent is obtained by subtracting the content of Parenthood from the Cartesian product of Person and Person
- The second iteration adds , aggiungiamo al valore precedente di Descendant grows, and NonDescendant should get smaller, but it **cannot be reduced**
- The final result is wrong, as anybody without a direct parent-child relationship would result as non-descendant

Problems with negation and recursion

- The solutioni s to stratify rule sets

Layer 1:

Descendant(X,Y) :- Parenthood(X,Y)

Descendant(X,Y) :- Descendant(X,Z), Parenthood(Z,Y)

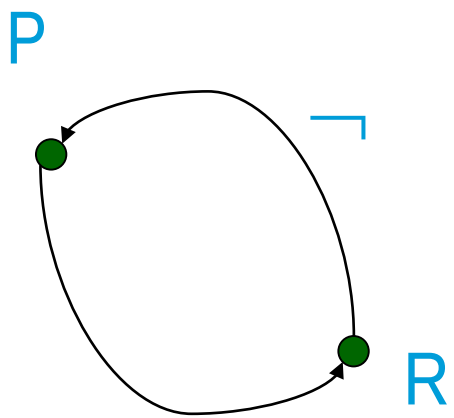
Layer 2:

NonDescendant(X,Y) :- Person(X,_,_), Person (Y,_,_), ¬ Descendant(X,Y)

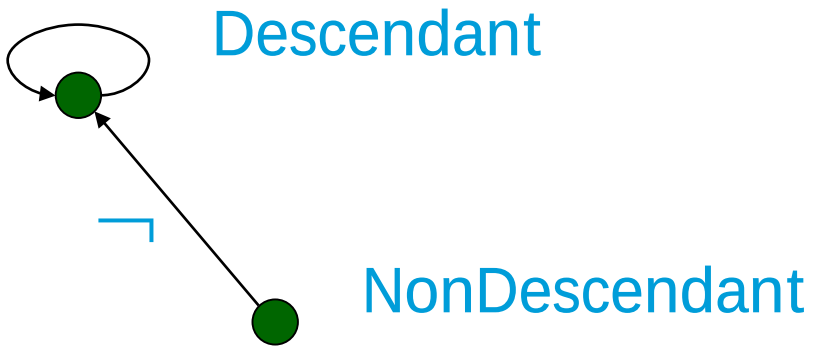
- We first compute the entire Descendant result, until the fixed point
- Then NonDescendant is computed by subtracting the final version from the Cartesian product of persons

Stratification

- Not all rule sets are stratifiable
- A graph of dependencies can be built, labeling with a minus the negative dependencies
- If the graph has no cycles with negated arcs, then the rule set is stratifiable



Non STRATIFIABLE

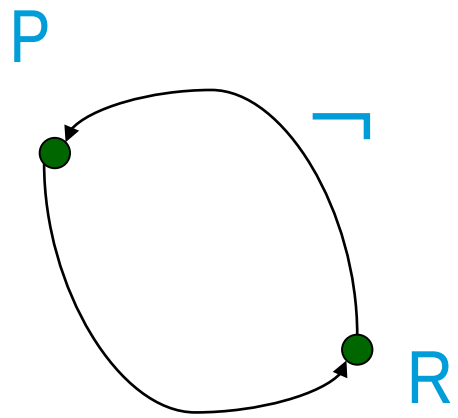


STRATIFIABLE

Correct Rules

R(Y) :- T(X,Y), ¬ P(Y) not OK

P(X) :- R(X),S(Y,X)



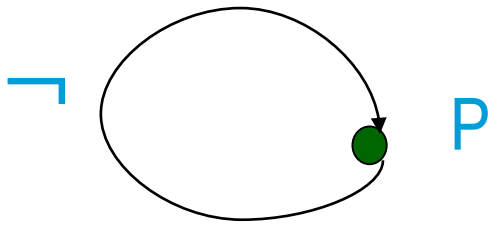
Correct rules

Another problem with negation:

P(X) :- ¬ P(X), Q(X)

EDB: Q(1), Q(2)

Initiallly: P = { }
step 1: P = {(1), (2)}
step 2: P = { }
step 3: P = {(1), (2)}, etc.,



A fixed point is never reached

Safe Rules (2/2)

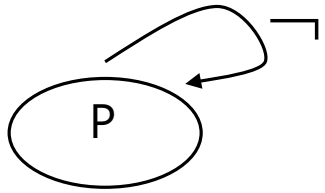
All variables that occur in a negated literal must also occur in a positive literal in the rule body

S(X) :- ¬ R(X) **not** OK

S(X) :- ¬ R(X), P(X) OK

- Negation must be ***stratified***
  in shiort: there can be **no cyclic dependencies** between negated literals

P(X) :- R(X), ¬ P(X) is not OK



Functions are dangerous too

We should always take care:

***n*(*X*+*1*) :- *n*(*X*)** the successor of a number is a number

***n*(1)** 1 is a number

***? - n*(*X*)** returns an infinite result!

Summary of Datalog «slang»

Standard Relational Model		**Datalog**
Relation	→	***Predicate (Literal)***
Attribute	→	***Argument***
Tuple	→	***Fact***
View	→	***Rule***
Query	→	***Goal***

Comparison of expressive powers

Datalog with safe negation is more expressive than Relational Algebra due to

- Recursion
- Capability of generating new data items
    - As long as safe, tratified rules do not generate infinite symbols

But… why formal languages?

- **Relational Algebra** is usefulto understand how the optimization of actual queries works in a DBMS
- **Datalog** bridges the worlds of databases, logic, and artificial intelligence
  - Data Mining, Data Analysis, Knowledge Discovery
  - Inference and self-correcting databases
  - … domain-specific expert systems

Our «classic» queries. Extract:

- *Students examined by prof. Meo*
- *The courses passed by Antonio*
- *The courses passed by Antonio, with date and grade*
- *The students who passed Math **or** Databases*
- *The students who passed both Math **and** Databases*
- *The students who passed Math **but not** Databases*
- *The students who passed Math and Databases on the same day*
- *The students who never got less than 28*
- *The students who got a 30 and never less than 28*
- *The first exam of each student*

Student

SId	Name	City	Dept
123	Carlo	Bologna	CS
415	Paola	Torino	CS
702	Antonio	Roma	Log

Exam

SId	CourseId	Date	Grade
123	1	7-9-23	30
123	2	8-1-24	28
702	2	7-9-23	25

Course

CourseId	Title	Teacher
1	Math	Barozzi
2	Databases	Meo